

## Diseño y Análisis de Algoritmos

### Ayudantía 3: Correctitud: Invariante de Loop

Ayud. Ferran Rojas Andreu

## Enunciado

Sea  $A = \langle a_0, \dots, a_{n-1} \rangle \in \mathbb{Z}^n$  con  $n \geq 1$ . Considere el siguiente algoritmo de *ordenación por inserción*, que ordena  $A$  en orden no decreciente:

INSERTION-SORT( $A$ )

```
1  for  $i = 1$  to  $n - 1$ 
2      key =  $A[i]$ 
3       $j = i - 1$ 
4      while  $j \geq 0$  and  $A[j] > \text{key}$ 
5           $A[j + 1] = A[j]$ 
6           $j = j - 1$ 
7       $A[j + 1] = \text{key}$ 
8  return  $A$ 
```

Demuestre la correctitud del algoritmo utilizando la técnica de *invariante de loop* sobre el bucle **for** exterior. Defina formalmente el invariante  $I(i)$  y pruebe las tres propiedades necesarias:

- (a) **Inicialización:**  $I(1)$  se cumple antes de la primera iteración.
- (b) **Mantenimiento:** si  $I(i)$  es verdadero al inicio de la iteración  $i$ , entonces  $I(i + 1)$  se cumple al inicio de la siguiente.
- (c) **Terminación:** al finalizar el bucle, la propiedad resultante implica que  $A$  está ordenado.

*Sugerencia:* el invariante debe capturar que el prefijo ya procesado  $A[0 \dots i - 1]$  está ordenado y que, además, el algoritmo conserva el conjunto de elementos del arreglo original.

## Enunciado 2: Ventana deslizante

Sea  $A = \langle a_1, \dots, a_n \rangle \in \mathbb{Z}^n$  y sea  $1 \leq k \leq n$ . Se desea encontrar la *máxima suma* de cualquier subarreglo contiguo de  $A$  de longitud *exactamente*  $k$ :

$$\max_{1 \leq i \leq n-k+1} \sum_{t=i}^{i+k-1} A[t].$$

Considere el siguiente algoritmo de *ventana deslizante* (sliding window), que reutiliza la suma de la ventana anterior en tiempo  $O(1)$  por desplazamiento:

SLIDINGWINDOWMAXSUM( $A[1..n]$ ,  $n$ ,  $k$ )

```

1  window  $\leftarrow \sum_{t=1}^k A[t]$ 
2  best  $\leftarrow$  window
3  for  $i = k + 1$  to  $n$ 
4      window  $\leftarrow$  window  $- A[i - k] + A[i]$ 
5      best  $\leftarrow$   $\max(\text{best}, \text{window})$ 
6  return best
```

Para el ejemplo

$$A = \langle 2, 1, 5, 1, 3, 2 \rangle, \quad k = 3,$$

determine:

- la ejecución paso a paso del algoritmo, mostrando el valor de **window** y **best** en cada iteración, y el valor retornado;
- demuestre la correctitud del algoritmo usando la técnica de *invariante de loop* sobre el bucle **for**: defina formalmente el invariante  $I(i)$  y pruebe la **inicialización**, el **mantenimiento** y la **terminación**;
- analice la complejidad temporal y espacial del algoritmo.

*Sugerencia:* el invariante debe relacionar **window** con la ventana que termina en la posición anterior  $i - 1$ , y **best** con el máximo de todas las ventanas ya recorridas  $A[1 \dots i - 1]$ .